Name : Tanmay Mahesh Ahuja **Semester :** V **Branch :** AI&DS

Roll No : 03 **Batch :** T11 **Date :** 20-07-2026

Experiment No. 02

Aim : To containerize applications using Docker.

LO Mapped : LO4

LO : Containerize, orchestrate and deploy applications using Docker, Docker Compose, Kubernetes and cloud infrastructure tools.

Theory :

1. What is Docker ?

Docker is the industry-standard, open-source platform designed to automate the deployment, scaling, and management of containerized applications. It provides a comprehensive ecosystem and tooling framework to build, ship, and run containers.

Core Docker Architecture & Components

- **Docker Daemon (dockerd):** The background service running on the host machine that manages building, running, and distributing Docker containers.
- **Docker Client (docker):** The primary command-line interface (CLI) that allows users to interact with the Docker Daemon.
- **Docker Image:** A read-only, immutable template containing the application source code, runtime, libraries, dependencies, and environment variables needed to run the application.
- **Docker Container:** A runnable instance of a Docker image. It is a live, isolated environment where the application actively executes.
- **Docker Registry (Docker Hub):** A centralized, cloud-based repository where Docker images are stored, shared, and distributed.
- **Dockerfile:** A plain-text configuration file containing a set of sequential instructions that the Docker Daemon reads to automatically build a custom Docker image.

2. What is Containerization ?

Containerization is a lightweight form of operating system-level virtualization. It involves packaging a software application and all of its necessary dependencies—including libraries, frameworks, configuration files, and runtime engines—into a single, isolated, and executable software unit called a container.

Unlike traditional hypervisor-based virtualization (Virtual Machines), which requires installing a heavy, full-fledged guest operating system for every application, containerization leverages the host system's operating system kernel. The container engine (like Docker) isolates the application's processes, CPU, and memory at the OS level.

Consider the deployment of a full-stack relational database application like StayHotels. Instead of manually installing Node.js, configuring Express MVC routing folders, and setting up the MySQL backend architecture on every new server or evaluator's machine, containerization wraps this entire ecosystem into a single deployable unit. This ensures the application behaves exactly the same way in a local VS Code environment as it does in a live production server.

Key Characteristics of Containerization:

- **Immutability:** Once a containerized environment is built, its core components do not change. This eliminates environmental inconsistencies.
- **Resource Efficiency:** Because they share the host OS kernel and do not require a hypervisor, containers consume significantly less RAM and CPU, allowing dozens of containers to run on a single host.
- **High Portability:** A containerized application can run on any system that supports the container runtime, regardless of the underlying hardware or operating system (Windows, Linux, macOS, or Cloud infrastructure).
- **Rapid Provisioning:** Containers can boot up in milliseconds since they do not need to boot an entire operating system.

3. Docker Images vs. Docker Containers

A common point of confusion in DevOps theory is the distinction between containerization (the concept), Docker images, and Docker containers.

To clarify: **Containerization** is the overarching virtualization methodology. **Docker images** and **Docker containers** are the specific, distinct technical components used to execute that methodology.

An image and a container represent two different phases of the application lifecycle: the *build* phase and the *run* phase.

Docker Image (The Blueprint)

A Docker Image is a read-only, static, and immutable file containing the source code, libraries, dependencies, tools, and other files needed for an application to run.

- **Nature:** It is essentially a snapshot of an environment at a specific point in time.
- **State:** It is completely stateless. It does not execute code; it simply stores the compiled files and instructions.
- **Storage:** Images are stored on physical disk space and can be pushed to or pulled from repositories like Docker Hub.

- **Analogy:** In object-oriented programming, an image is equivalent to a **Class**. In architecture, it is the **Blueprint** of a house.

Docker Container (The Active Instance)

A Docker Container is the live, running, active instance of a Docker Image.

- **Nature:** When the Docker engine executes an image using the `docker run` command, it adds a thin, read-write layer on top of the read-only image. This creates the container.
- **State:** It is stateful and dynamic. It actively consumes host resources (CPU, Memory, Network ports) to execute the application code.
- **Storage:** Containers are ephemeral by default. If a container is deleted, any data written to its read-write layer is destroyed unless explicitly saved using Docker Volumes.
- **Analogy:** Continuing the object-oriented programming analogy, a container is the **Object** instantiated from the Class. In architecture, it is the actual physical **House** built from the blueprint.

Detailed Comparison Table

Feature	Docker Image	Docker Container
Definition	A read-only template containing application dependencies and configurations.	A runnable, active instance of a Docker image.
Lifecycle State	Static / Passive (Resting state).	Dynamic / Active (Running state).
Mutability	Immutable. Once built, it cannot be modified. Updates require building a new image.	Mutable. The application inside can create, modify, and delete temporary files while running.
Resource Consumption	Consumes only disk storage space.	Consumes active system RAM, CPU, and network bandwidth.
Creation Command	Created using `docker build` (from a Dockerfile) or `docker commit`.	Created using `docker run` or `docker start`.
Portability	Highly portable; easily shared via registries (e.g., `docker push`).	Bound to the host machine where it is actively running.
Quantity	A single image can exist on a system.	Multiple independent containers can be launched simultaneously from that single image.

4. Practical Execution and Command Line Analysis

During this laboratory experiment, a series of operations were executed via the Docker CLI to demonstrate the lifecycle of container management. Below is a detailed theoretical breakdown of each step.

4.1 Environment Verification and System State

Before deploying applications, it is necessary to verify the operational state of the container engine.

- docker --version: This command checks the currently installed version of the Docker client, ensuring compatibility with required features. The experiment verified the environment was running Docker version 29.2.1.
- docker info: This command provides a system-wide overview of the Docker installation, including the number of running/stopped containers, storage drivers, and host memory configuration.

4.2 Image Management and Retrieval

Images act as the blueprint for containers. Managing these images efficiently is a core DevOps task.

- docker images: This command queries the local host system to list all currently downloaded images. In the experiment, the local repository contained images such as 'hello-world', 'myredis', 'redis', and 'welcome-to-docker'.
- docker search <term>: This command queries the public Docker Hub registry for images matching the provided term. Executing docker search ubuntu retrieved a list of available Ubuntu-based images, displaying their descriptions, community star ratings, and official status.
- **Applying Filters:** To refine search results and locate highly reputable images, the --filter flag is utilized. The command docker search --filter=stars=120 ubuntu successfully narrowed the repository query to only display Ubuntu images with a minimum of 120 community stars.
- docker pull <image_name>: Once a suitable image is identified, this command downloads it from the registry to the local host machine. The experiment successfully downloaded the latest official Ubuntu image using docker pull ubuntu.
- docker rmi <image_name>: This command removes an image from the local storage to free up disk space. (Note: An image cannot be removed if a container, even a stopped one, is currently referencing it unless the force flag -f is applied).
- docker image prune: A maintenance command used to remove all dangling images (layers that have no relationship to any tagged images), optimizing host storage.

4.3 Container Lifecycle Execution

Creating, running, and stopping containers represents the core operational phase of the experiment.

- docker run: This is the primary command used to create and start a new container from an image. It combines the functions of creating the container and immediately starting it.

- **Interactive Execution (-it):** The flags -i (interactive) and -t (pseudo-TTY) are critical when a user needs to interact with the container's shell. The command docker run -it ubuntu /bin/bash launched an Ubuntu container and immediately attached the user's terminal to the container's internal bash shell. This allows direct command execution inside the isolated environment, which was terminated using the exit command.
- **Detached Execution (-d):** For servers and background services (like databases), containers must run in the background. Adding -d (e.g., docker run -d redis) runs the container detached from the current terminal session.
- **Port Mapping (-p):** To access a containerized application from the outside world, internal ports must be mapped to the host's ports (e.g., -p 6380:6379).
- **Naming (--name):** Assigning a custom, human-readable name to a container (e.g., --name my_lab_container) simplifies lifecycle management compared to using randomly generated hash IDs.
- docker ps: This command lists all currently running containers. Appending the --all or -a flag lists every container on the system, regardless of its current state (running, stopped, or exited).
- docker stop <container_name>: This sends a graceful termination signal (SIGTERM) to the running container, allowing processes to shut down safely.
- docker rm <container_name>: This permanently deletes a stopped container from the host system.

4.4 Monitoring, Inspection, and Debugging

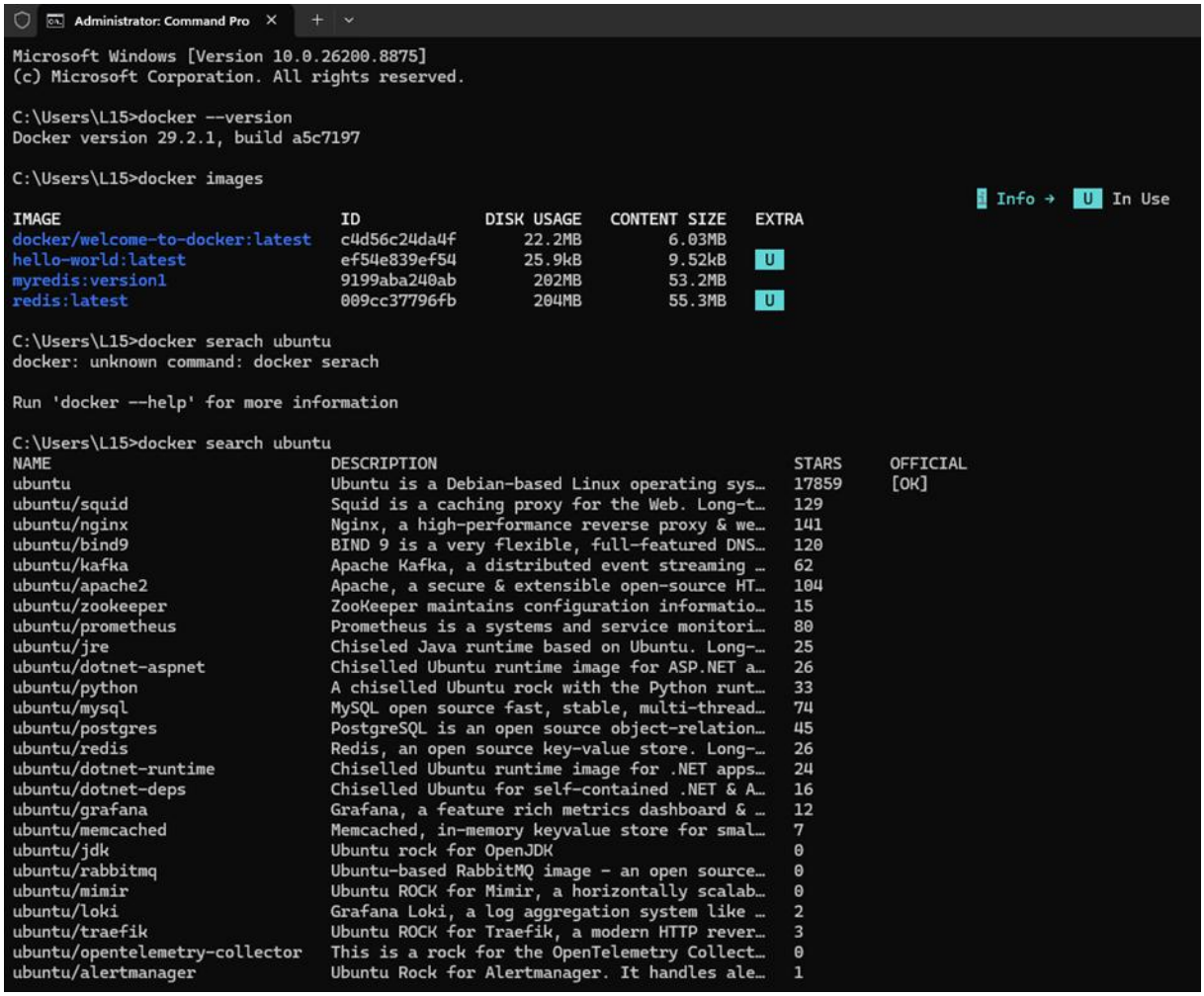
In Agile methodologies, rapid debugging is essential. Docker provides built-in tools for monitoring application health.

- docker container stats: This command provides a live, continuously updating stream of resource usage metrics (CPU, Memory, Network I/O) for all running containers, acting similarly to a task manager.
- docker logs <container_name>: This command fetches the standard output (STDOUT) and standard error (STDERR) logs generated by the application running inside the container, crucial for troubleshooting headless or detached services.
- docker inspect <container_name>: This generates a comprehensive JSON array detailing the low-level configuration of the container, including assigned IP addresses, mounted volumes, and environment variables.
- docker exec -it <container_name> /bin/bash: Unlike docker run which starts a *new* container, docker exec runs a new command (like opening a bash shell) inside an *already running* container. This is vital for live debugging.

4.5 Advanced Concepts: Building and Persistence

- **Building Custom Images (docker build):** By defining requirements in a Dockerfile, users can automate the creation of custom environments. The command docker build -t <image_name> . commands the daemon to read the Dockerfile in the current directory (.) and compile a new image tagged with the specified name.
- **Data Volumes (docker volume create & -v):** By default, container data is ephemeral; if the container is removed, the data is lost. In DevOps, databases require persistence. Docker Volumes are isolated storage mechanisms managed by Docker that persist independently of container lifecycles. Using the -v flag during a run command mounts a volume into the container, ensuring critical application data survives container destruction.

Implementation



```
C:\Users\L15>docker search --filter=stars=120 ubuntu
NAME              DESCRIPTION                                     STARS     OFFICIAL
ubuntu            Ubuntu is a Debian-based Linux operating sys…   17859     [OK]
ubuntu/squid      Squid is a caching proxy for the Web. Long-t…   129
ubuntu/nginx      Nginx, a high-performance reverse proxy & we…   141
ubuntu/bind9      BIND 9 is a very flexible, full-featured DNS…   120

C:\Users\L15>docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
a3679419df18: Pull complete
ed819469700f: Pull complete
e16351a257e4: Download complete
Digest: sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest

C:\Users\L15>docker images
                                                                                  i Info →  U In Use
IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/welcome-to-docker:latest      c4d56c24da4f       22.2MB         6.03MB
hello-world:latest                   ef54e839ef54       25.9kB         9.52kB    U
myredis:version1                     9199aba240ab        202MB         53.2MB
redis:latest                         009cc37796fb        204MB         55.3MB    U
ubuntu:latest                        3131b4cc82a7        161MB         45.3MB

C:\Users\L15>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\L15>docker run -it ubuntu /bin/bash
root@80c22874ea4d:/# exit
exit
```

```
C:\Users\L15>docker info
Client:
 Version:    29.2.1
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  agent: create or run AI agents (Docker Inc.)
    Version:  v1.27.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-agent.exe
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.18.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.31.1-desktop.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v5.0.2
    Path:     C:\Program Files\Docker\cli-plugins\docker-compose.exe
  debug: Get a shell into any image or container (Docker Inc.)
    Version:  0.0.47
    Path:     C:\Program Files\Docker\cli-plugins\docker-debug.exe
  desktop: Docker Desktop commands (Docker Inc.)
    Version:  v0.3.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-desktop.exe
  extension: Manages Docker extensions (Docker Inc.)
    Version:  v0.2.31
    Path:     C:\Program Files\Docker\cli-plugins\docker-extension.exe
  init: Creates Docker-related starter files for your project (Docker Inc.)
    Version:  v1.4.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-init.exe
  mcp: Docker MCP Plugin (Docker Inc.)
    Version:  v0.40.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-mcp.exe
  model: Docker Model Runner (Docker Inc.)
    Version:  v1.1.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-model.exe
  offload: Docker Offload (Docker Inc.)
    Version:  v0.5.56
    Path:     C:\Program Files\Docker\cli-plugins\docker-offload.exe
  pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
    Version:  v0.0.24
    Path:     C:\Program Files\Docker\cli-plugins\docker-pass.exe
  sandbox: Docker Sandbox (Docker Inc.)
    Version:  v0.12.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-sandbox.exe
  sbom: View the packaged-based Software Bill Of Materials (SBOM) for an image (Anchore Inc.)
    Version:  0.6.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-sbom.exe
  scout: Docker Scout (Docker Inc.)
    Version:  v1.20.0
```

```
Server:
 Containers: 4
  Running: 0
  Paused: 0
  Stopped: 4
 Images: 5
 Server Version: 29.2.1
 Storage Driver: overlayfs
  driver-type: io.containerd.snapshotter.v1
 Logging Driver: json-file
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS


C:\Users\L15>docker run --name my_lab_redis -d redis
0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb

C:\Users\L15>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED                  STATUS         PORTS      NAMES
0b05c060a9f5   redis     "docker-entrypoint.s…"   Less than a second ago   Up 25 seconds   6379/tcp   my_lab_redis

C:\Users\L15>docker run --name my_mapped_redis -d -p 6380:6379 redis
0925a4ada1b2abb506f5f42f2334524f69ec6a399c9a5904e88854f4867c2aba7

C:\Users\L15>docker logs my_lab_redis
Starting Redis Server
1:C 20 Jul 2026 10:22:12.909 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
1:C 20 Jul 2026 10:22:12.909 * Redis version=8.6.2, bits=64, commit=00000000, modified=1, pid=1, just started
1:C 20 Jul 2026 10:22:12.909 * Configuration loaded
1:M 20 Jul 2026 10:22:12.911 * monotonic clock: POSIX clock_gettime
1:M 20 Jul 2026 10:22:12.912 * Running mode=standalone, port=6379.
1:M 20 Jul 2026 10:22:12.914 * <bf> RedisBloom version 8.6.0 (Git=unknown)
1:M 20 Jul 2026 10:22:12.914 * <bf> Registering configuration options: [
1:M 20 Jul 2026 10:22:12.914 * <bf>     { bf-error-rate       :      0.01 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { bf-initial-size     :       100 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { bf-expansion-factor :         2 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { cf-bucket-size      :         2 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { cf-initial-size     :      1024 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { cf-max-iterations   :        20 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { cf-expansion-factor :         1 }
1:M 20 Jul 2026 10:22:12.914 * <bf>     { cf-max-expansions   :        32 }
1:M 20 Jul 2026 10:22:12.914 * <bf> ]
1:M 20 Jul 2026 10:22:12.914 * Module 'bf' loaded from /usr/local/lib/redis/modules//redisbloom.so
1:M 20 Jul 2026 10:22:12.933 * <search> Redis version found by RedisSearch : 8.6.2 - oss
1:M 20 Jul 2026 10:22:12.933 * <search> RediSearch version 8.6.0 (Git=7782b97)
1:M 20 Jul 2026 10:22:12.933 * <search> Low level api version 1 initialized successfully
1:M 20 Jul 2026 10:22:12.933 * <search> gc: ON, prefix min length: 2, min word length to stem: 4, prefix max expansions: 200, query timeout (ms): 500, timeout policy: return, oom policy: return, cursor read si
ze: 1000, cursor max idle (ms): 300000, max doctable size: 1000000, max number of search results:  1000000, default scorer: BM25STD,
1:M 20 Jul 2026 10:22:12.934 * <search> Initialized thread pools!
1:M 20 Jul 2026 10:22:12.934 * <search> Disabled workers threadpool of size 0
1:M 20 Jul 2026 10:22:12.934 * <search> Subscribe to config changes
1:M 20 Jul 2026 10:22:12.934 * <search> Subscribe to cluster slot migration events
```

```
1:M 20 Jul 2026 10:22:12.934 * <search> Initialized thread pools!
1:M 20 Jul 2026 10:22:12.934 * <search> Disabled workers threadpool of size 0
1:M 20 Jul 2026 10:22:12.934 * <search> Subscribe to config changes
1:M 20 Jul 2026 10:22:12.934 * <search> Subscribe to cluster slot migration events
1:M 20 Jul 2026 10:22:12.934 * <search> Enabled role change notification
1:M 20 Jul 2026 10:22:12.935 * <search> Cluster configuration: AUTO partitions, type: 0, coordinator timeout: 0ms
1:M 20 Jul 2026 10:22:12.935 * Module 'search' loaded from /usr/local/lib/redis/modules//redisearch.so
1:M 20 Jul 2026 10:22:12.936 * <timeseries> RedisTimeSeries version 80600, git_sha=05fd355db7f48676861dc4c17d19c8c1ca74c0154
1:M 20 Jul 2026 10:22:12.936 * <timeseries> Redis version found by RedisTimeSeries : 8.6.2 - oss
1:M 20 Jul 2026 10:22:12.936 * <timeseries> Registering configuration options: [
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-compaction-policy   :              }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-num-threads         :            3 }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-retention-policy    :            0 }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-duplicate-policy    :        block }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-chunk-size-bytes    :         4096 }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-encoding            :   compressed }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-ignore-max-time-diff:            0 }
1:M 20 Jul 2026 10:22:12.936 * <timeseries>     { ts-ignore-max-val-diff :     0.000000 }
1:M 20 Jul 2026 10:22:12.936 * <timeseries> ]
1:M 20 Jul 2026 10:22:12.937 * <timeseries> Detected redis oss
1:M 20 Jul 2026 10:22:12.937 * <timeseries> Subscribe to ASM events
1:M 20 Jul 2026 10:22:12.937 * <timeseries> Enabled diskless replication
1:M 20 Jul 2026 10:22:12.937 * Module 'timeseries' loaded from /usr/local/lib/redis/modules//redistimeseries.so
1:M 20 Jul 2026 10:22:12.942 * <ReJSON> Created new data type 'ReJSON-RL'
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> version: 80600 git sha: unknown branch: unknown
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Exported RedisJSON_V1 API
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Exported RedisJSON_V2 API
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Exported RedisJSON_V3 API
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Exported RedisJSON_V4 API
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Exported RedisJSON_V5 API
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Exported RedisJSON_V6 API
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Enabled diskless replication
1:M 20 Jul 2026 10:22:12.943 * <ReJSON> Initialized shared string cache, thread safe: true.
1:M 20 Jul 2026 10:22:12.943 * Module 'ReJSON' loaded from /usr/local/lib/redis/modules//rejson.so
1:M 20 Jul 2026 10:22:12.943 * <search> Acquired RedisJSON_V6 API
1:M 20 Jul 2026 10:22:12.944 * Server initialized
1:M 20 Jul 2026 10:22:12.944 * Ready to accept connections tcp
1:M 20 Jul 2026 10:22:12.944 # WARNING: Redis does not require authentication and is not protected by network restrictions. Redis will accept connections from any IP address on any network interface.

C:\Users\L15>docker inspect my_lab_redis
[
    {
        "Id": "0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb",
        "Created": "2026-07-20T10:22:12.673590477Z",
        "Path": "docker-entrypoint.sh",
        "Args": [
            "redis-server"
        ],
```

```
        ],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 542,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-07-20T10:22:12.718942097Z",
            "FinishedAt": "0001-01-01T00:00:00Z"
        },
        "Image": "sha256:009cc37796fbdbe1b631b4cc0582bed167e5e403ed8bcd06f77eb6cb5aeb6f93",
        "ResolvConfPath": "/var/lib/docker/containers/0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb/resolv.conf",
        "HostnamePath": "/var/lib/docker/containers/0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb/hostname",
        "HostsPath": "/var/lib/docker/containers/0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb/hosts",
        "LogPath": "/var/lib/docker/containers/0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb/0b05c060a9f58ddc83003beac92d9faa8b98e2c97c936ed9c1b2fcb9ad4dcfdb-json.log",
        "Name": "/my_lab_redis",
        "RestartCount": 0,
        "Driver": "overlayfs",
        "Platform": "linux",
        "MountLabel": "",
        "ProcessLabel": "",
        "AppArmorProfile": "",
        "ExecIDs": null,
        "HostConfig": {
            "Binds": null,
            "ContainerIDFile": "",
            "LogConfig": {
                "Type": "json-file",
                "Config": {}
            },
            "NetworkMode": "bridge",
            "PortBindings": {},
            "RestartPolicy": {
                "Name": "no",
                "MaximumRetryCount": 0
            },
            "AutoRemove": false,
            "VolumeDriver": "",
            "VolumesFrom": null,
            "ConsoleSize": [
                51,
                138
            ],
            "CapAdd": null,
            "CapDrop": null,
            "CgroupnsMode": "private",
            "Dns": null,
```

```
"CgroupnsMode": "private",
"Dns": null,
"DnsOptions": [],
"DnsSearch": [],
"ExtraHosts": null,
"GroupAdd": null,
"IpcMode": "private",
"Cgroup": "",
"Links": null,
"OomScoreAdj": 0,
"PidMode": "",
"Privileged": false,
"PublishAllPorts": false,
"ReadonlyRootfs": false,
"SecurityOpt": null,
"UTSMode": "",
"UsernsMode": "",
"ShmSize": 67108864,
"Runtime": "runc",
"Isolation": "",
"CpuShares": 0,
"Memory": 0,
"NanoCpus": 0,
"CgroupParent": "",
"BlkioWeight": 0,
"BlkioWeightDevice": [],
"BlkioDeviceReadBps": [],
"BlkioDeviceWriteBps": [],
"BlkioDeviceReadIOps": [],
"BlkioDeviceWriteIOps": [],
"CpuPeriod": 0,
"CpuQuota": 0,
"CpuRealtimePeriod": 0,
"CpuRealtimeRuntime": 0,
"CpusetCpus": "",
"CpusetMems": "",
"Devices": [],
"DeviceCgroupRules": null,
"DeviceRequests": null,
"MemoryReservation": 0,
"MemorySwap": 0,
"MemorySwappiness": null,
"OomKillDisable": null,
"PidsLimit": null,
"Ulimits": [],
"CpuCount": 0,
"CpuPercent": 0,
"IOMaximumIOps": 0,
"IOMaximumBandwidth": 0,
"MaskedPaths": [
```

```
        "MaskedPaths": [
            "/proc/acpi",
            "/proc/asound",
            "/proc/interrupts",
            "/proc/kcore",
            "/proc/keys",
            "/proc/latency_stats",
            "/proc/sched_debug",
            "/proc/scsi",
            "/proc/timer_list",
            "/proc/timer_stats",
            "/sys/devices/virtual/powercap",
            "/sys/firmware"
        ],
        "ReadonlyPaths": [
            "/proc/bus",
            "/proc/fs",
            "/proc/irq",
            "/proc/sys",
            "/proc/sysrq-trigger"
        ]
    },
    "Storage": {
        "RootFS": {
            "Snapshot": {
                "Name": "overlayfs"
            }
        }
    },
    "Mounts": [],
    "Config": {
        "Hostname": "0b05c060a9f5",
        "Domainname": "",
        "User": "",
        "AttachStdin": false,
        "AttachStdout": false,
        "AttachStderr": false,
        "ExposedPorts": {
            "6379/tcp": {}
        },
        "Tty": false,
        "OpenStdin": false,
        "StdinOnce": false,
        "Env": [
            "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
        ],
        "Cmd": [
            "redis-server"
        ],
        "Image": "redis",
```

```
      "Image": "redis",
      "Volumes": null,
      "WorkingDir": "/data",
      "Entrypoint": [
          "docker-entrypoint.sh"
      ],
      "Labels": {},
      "StopTimeout": 1
  },
  "NetworkSettings": {
      "SandboxID": "283db676f7387b958d610f44f991278a3d428d04a916ef33010047d52d7c9f99",
      "SandboxKey": "/var/run/docker/netns/283db676f738",
      "Ports": {
          "6379/tcp": null
      },
      "Networks": {
          "bridge": {
              "IPAMConfig": null,
              "Links": null,
              "Aliases": null,
              "DriverOpts": null,
              "GwPriority": 0,
              "NetworkID": "ca0543cf5dac9e6195a9e279fdb5a4048957b9ccc1aff3c976da008e7400abdf",
              "EndpointID": "bdd3b732a79179852c71990f2b342e3d9ac4d592ed19c1096d65950a3d61b615",
              "Gateway": "172.17.0.1",
              "IPAddress": "172.17.0.2",
              "MacAddress": "c6:85:95:c8:d8:a9",
              "IPPrefixLen": 16,
              "IPv6Gateway": "",
              "GlobalIPv6Address": "",
              "GlobalIPv6PrefixLen": 0,
              "DNSNames": null
          }
      }
  },
  "ImageManifestDescriptor": {
      "mediaType": "application/vnd.oci.image.manifest.v1+json",
      "digest": "sha256:17a235ee842cd21d8bcb7f11a49fb177cbeaf5537996304a0089e94412de2ed5",
      "size": 2286,
      "annotations": {
          "com.docker.official-images.bashbrew.arch": "amd64",
          "org.opencontainers.image.base.digest": "sha256:edc9450a9fe37d30b508808052f8d0564e3ed9eaf565e043c6f5632957f7381e",
          "org.opencontainers.image.base.name": "debian:trixie-slim",
          "org.opencontainers.image.created": "2026-03-24T18:12:25Z",
          "org.opencontainers.image.revision": "8c81e2a44cae9258294718d767ce594e5cbbf20e",
          "org.opencontainers.image.source": "https://github.com/redis/docker-library-redis.git#8c81e2a44cae9258294718d767ce594e5cbbf20e:debian",
          "org.opencontainers.image.url": "https://hub.docker.com/_/redis",
          "org.opencontainers.image.version": "8.6.2"
      },
      "platform": {
          "architecture": "amd64",
          "os": "linux"
      }
  }
}
]

C:\Users\L15>docker stop my_lab_redis
my_lab_redis

C:\Users\L15>docker stop my_mapped_redis
my_mapped_redis

C:\Users\L15>docker ps --all
CONTAINER ID   IMAGE         COMMAND                  CREATED                  STATUS                      PORTS     NAMES
0925a4ada1b2   redis         "docker-entrypoint.s…"   Less than a second ago   Exited (0) 8 seconds ago              my_mapped_redis
0b05c060a9f5   redis         "docker-entrypoint.s…"   Less than a second ago   Exited (0) 16 seconds ago             my_lab_redis
80c22874ea4d   ubuntu        "/bin/bash"              Less than a second ago   Exited (0) 4 minutes ago              relaxed_almeida
83ab4fe6a825   redis         "docker-entrypoint.s…"   3 months ago             Exited (0) 3 months ago               crazy_agnesi
b7a18b45f217   redis         "docker-entrypoint.s…"   3 months ago             Exited (0) 3 months ago               awesome_allen
222596f84fc0   hello-world   "/hello"                 4 months ago             Exited (0) 4 months ago               gracious_wilson

C:\Users\L15>docker rm my_lab_redis
my_lab_redis

C:\Users\L15>docker rm my_mapped_redis
my_mapped_redis

C:\Users\L15>docker rmi hello-world
Error response from daemon: conflict: unable to delete hello-world:latest (must be forced) - container 222596f84fc0 is using its referenced image ef54e839ef54

C:\Users\L15>docker image prune
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B

C:\Users\L15>docker rm 222596f84fc0
222596f84fc0

C:\Users\L15>docker rmi hello-world
Untagged: hello-world:latest
Deleted: sha256:ef54e839ef541993b4e87f25e752f7cf4238fa55f017957c2eb44077083d7a6a

C:\Users\L15>echo FROM ubuntu > Dockerfile

C:\Users\L15>echo CMD ["echo", "My custom image build was successful!"] >> Dockerfile

C:\Users\L15>docker build -t my_custom_image .
[+] Building 0.3s (5/5) FINISHED                                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                          0.0s
```

```
C:\Users\L15>docker build -t my_custom_image .
[+] Building 0.3s (5/5) FINISHED                                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                0.0s
 => => transferring dockerfile: 107B                                                                                                0.0s
 => [internal] load metadata for docker.io/library/ubuntu:latest                                                                    0.0s
 => [internal] load .dockerignore                                                                                                   0.0s
 => => transferring context: 2B                                                                                                     0.0s
 => [1/1] FROM docker.io/library/ubuntu:latest@sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb              0.0s
 => => resolve docker.io/library/ubuntu:latest@sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb              0.0s
 => exporting to image                                                                                                              0.1s
 => => exporting layers                                                                                                             0.0s
 => => exporting manifest sha256:165c2f52bc45dd0b98877fc9437e2833db17f21f9d80b3c7254f94aca5eccb60                                   0.0s
 => => exporting config sha256:f36bef7221b0c2d4123a6fdfdbd626e405ff1b44e13457569a1e5e87b6d0a85f                                     0.0s
 => => exporting attestation manifest sha256:7227e9785405e65c044d93fbae0d3f380931ea1ce7c459300f9bec5251ce26bc                       0.0s
 => => exporting manifest list sha256:76ac716dc74323242ee0613826fcaff286c99740760323c0cedcf6e243623269                              0.0s
 => => naming to docker.io/library/my_custom_image:latest                                                                           0.0s
 => => unpacking to docker.io/library/my_custom_image:latest                                                                        0.0s

C:\Users\L15>docker run my_custom_image
My custom image build was successful!

C:\Users\L15>docker run -d --name live_test_container ubuntu sleep 3600
7e6d5b1ab280e7d3b6270f4605fb0159b24daf16980feb5b7af71173b86f81ca

C:\Users\L15>docker exec -it live_test_container /bin/bash
root@7e6d5b1ab280:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@7e6d5b1ab280:/# pwd
/
root@7e6d5b1ab280:/# mkdir trial
root@7e6d5b1ab280:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  trial  usr  var
root@7e6d5b1ab280:/# cd trial
root@7e6d5b1ab280:/trial# exit
exit

C:\Users\L15>docker volume create my_database_data
my_database_data

C:\Users\L15>docker run -d --name my_mysql -v my_database_data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=root mysql
Unable to find image 'mysql:latest' locally
latest: Pulling from library/mysql
81b3a6a3bce8: Pull complete
e0607e526570: Pull complete
5ac73e0e3f6a: Pull complete
bb49c3208fe7: Pull complete
61d49498d3da: Pull complete
c2fb254fc2f4: Pull complete
08b808cb6d40: Pull complete
1576763b528a: Pull complete
e21df1635f12: Pull complete
```

```
C:\Users\L15>docker run -d --name my_mysql -v my_database_data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=root mysql
Unable to find image 'mysql:latest' locally
latest: Pulling from library/mysql
81b3a6a3bce8: Pull complete
e0607e526570: Pull complete
5ac73e0e3f6a: Pull complete
bb49c3208fe7: Pull complete
61d49498d3da: Pull complete
c2fb254fc2f4: Pull complete
08b808cb6d40: Pull complete
1576763b528a: Pull complete
e21df1635f12: Pull complete
88745f4755f0: Pull complete
a701ef10ab46: Download complete
1f9f19898688: Download complete
Digest: sha256:8fdf311514c91fa5014e93e98e19d7f5d9eb162a462c13100c956dacc278ee21
Status: Downloaded newer image for mysql:latest
97a63a9a708641ad3e6de14efb10431d8cd84f05b5a4765ea105884c80e12f9a

C:\Users\L15>docker stop live_test_container my_mysql
live_test_container
my_mysql

C:\Users\L15>docker rm live_test_container my_mysql
live_test_container
my_mysql

C:\Users\L15>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\L15>
```

Conclusion :

In conclusion, this experiment successfully demonstrated the practical application of containerization and environment isolation using the Docker framework. By managing official image repositories, executing interactive and detached container lifecycles, and establishing host-port mapping, the core competencies of consistent application deployment outlined in Learning Outcome 4 (LO4) were achieved. Furthermore, simulating container resource

monitoring and system inspections underscored the importance of environment reproducibility and infrastructure immutability within collaborative Agile and DevOps engineering environments.

LO4 mapped successfully.